SOFTWARE REQUIREMENTS SPECIFICATION

Recall It

March 2026

CSE 3206 — Software Engineering

Team Members

Ajwad 024

Nusair 052

Shammi 138

1. Introduction

1.1 Purpose

This document outlines what Recall It needs to do and how it should behave. Recall It is a flashcard app that uses the FSRS scheduling algorithm to show you the right cards at the right time — so you study smarter, not more.

It's the main reference for the team throughout design, development, and testing.

1.2 Intended Audience

This document is for everyone involved in building Recall It:

- Developers — to know what to build.

- Testers — to know what to verify.
- Project Managers — to track progress and scope.
- Designers — to understand how users interact with the app.
- Instructors — to evaluate what we're building.

1.3 Product Scope

Recall It is a web-based flashcard app (with mobile support planned). The main idea is simple: instead of drilling random cards, the app figures out exactly when you're about to forget something and shows it to you right then. That's powered by FSRS, a modern open-source scheduling algorithm.

Our goals:

- Build a cleaner, more modern alternative to Anki.
- Use FSRS v5 as the scheduling engine.
- Help people actually remember what they study.
- Support decks, card creation, and review sessions with live progress feedback.

1.4 Risk Definitions

Here are the main things that could go wrong and how we plan to handle them:

Risk	Description	Mitigation
Data Loss	User card/deck data or FSRS scheduling parameters are lost due to system failure.	Regular backups; export to .apkg/.csv.
Incorrect Scheduling	A bug in the FSRS implementation produces incorrect intervals, degrading user retention.	Unit tests against official FSRS reference implementation.
Unauthorized Access	Another user accesses or modifies another user's decks or review history.	Authenticated sessions; server-side authorization on all endpoints.
Poor Performance	High latency during review sessions breaks the learning flow and frustrates users.	Target < 300ms response; pre-fetch next card.

2. Overall Description

2.1 User Classes and Characteristics

Primary Users — Learners

Everyday learners — students, language learners, anyone trying to memorize things. They use the app daily and don't need to understand how the algorithm works. It just needs to work.

Secondary Users — Content Creators

Teachers or advanced users who build and share decks. They'll use the richer editing features like images and audio.

System Administrators

Technical staff who keep the backend running. They don't use the main app — they manage servers and user accounts.

2.2 User Needs

- Learners want a smooth daily review — no fuss, just the cards they need.
- Learners want to trust that the app is actually helping them, not just showing random cards.
- Content creators need a good editor — text, images, audio, LaTeX, fill-in-the-blank cards.
- Everyone needs their data to be safe and easy to export.
- The app should work well on phones and desktops.

2.3 Operating Environment

- Frontend: Works in all modern browsers (Chrome, Firefox, Safari, Edge).
- Backend: REST API deployed on a cloud platform or self-hosted server.
- Database: PostgreSQL for storing everything, Redis for caching.
- Mobile: Responsive design first; native app is a stretch goal.
- Offline: Users can review without internet; changes sync when they reconnect.

2.4 Constraints

- The FSRS implementation must match the official v5 spec exactly — same inputs, same outputs.
- User data must be stored securely and follow data protection laws (e.g. GDPR).
- We must support importing Anki's .apkg format so people can bring their existing decks.
- The app should run well on low-end phones, not just high-end devices.

2.5 Assumptions

- Users have internet for the initial sync. Offline mode is a backup, not the main way to use the app.
- FSRS v5 parameters won’t change significantly while we’re building.
- Users know what flashcards are, but probably haven’t heard of spaced repetition.
- Media files (images, audio) will be stored in a cloud bucket like AWS S3.

3. Requirements

3.1 Functional Requirements

Each feature is written as a user story: who needs it, what they want, and why. Each story has a checklist of what “done” looks like.

3.1.1 User Authentication

US-01	As a **new user**, I want to *register an account with email and password* so that I can save my decks and review history securely.
Acceptance Criteria	Registration form accepts email, username, and password. Password must be at least 8 characters with one uppercase and one number. A verification email is sent upon registration. Duplicate email addresses are rejected with a clear error message.

US-02	As a **registered user**, I want to *log in with my email and password* so that I can access my personal decks and review schedule.
Acceptance Criteria	Login form accepts email and password. Valid credentials redirect the user to their dashboard. Invalid credentials show an error without revealing which field is wrong. A JWT or session token is issued upon successful login.

US-03	As a **logged-in user**, I want to *reset my password via email* so that I can regain access if I forget my credentials.
Acceptance Criteria	A 'Forgot Password' link is present on the login page. User receives an email with a time-limited (1 hour) reset link. After reset, the old password no longer works.

3.1.2 Deck Management

US-04	As a **learner**, I want to *create a new deck with a name and optional description* so that I can organize my study material by topic.
Acceptance Criteria	A 'New Deck' button is visible on the dashboard. Name field is required; description is optional. Deck is saved and immediately visible in the deck list. Deck names must be unique per user.

US-05	As a **learner**, I want to *create sub-decks nested inside a parent deck* so that I can

	organize complex subjects hierarchically (e.g., Japanese > Hiragana).
Acceptance Criteria	User can select a parent deck when creating a new deck. Sub-decks appear as indented children in the deck list. Reviewing a parent deck includes all cards from its sub-decks.

US-06	As a **learner**, I want to *rename, edit, or delete a deck* so that I can keep my deck library organized and up to date.
Acceptance Criteria	Edit and delete options are accessible from each deck's context menu. Deleting a deck with cards requires a confirmation prompt. Deleting a deck also deletes all its cards and FSRS scheduling data.

US-07	As a **learner**, I want to *import an Anki package (.apkg file)* so that I can migrate my existing Anki decks without re-creating cards manually.
Acceptance Criteria	An 'Import' button accepts .apkg files. Cards, media, and deck structure are correctly imported. FSRS parameters are initialized fresh (Anki SM-2 state is not carried over). Import errors (malformed file, unsupported media) are reported clearly.

US-08	As a **learner**, I want to *export a deck as a .apkg or .csv file* so that I can back up my data or share it with others.
Acceptance Criteria	Export option is available per deck. CSV export includes front, back, and tags. .apkg export includes media files.

3.1.3 Card Creation and Editing

US-09	As a **content creator**, I want to *create a basic flashcard with a front (question) and back (answer)* so that I can add study material to a deck.
Acceptance Criteria	Card editor has clearly labeled Front and Back fields. Cards are saved and immediately available in the deck. Empty front or back fields are not accepted.

US-10	As a **content creator**, I want to *add images, audio, and LaTeX to card fields* so that I can create rich, multimedia flashcards for complex subjects.
Acceptance Criteria	Rich text editor supports image upload (PNG, JPEG, GIF, WebP). Audio files (MP3, OGG) can be attached and play during review. LaTeX expressions render correctly in both the editor and review view.

US-11	As a **content creator**, I want to *create a cloze deletion card (fill-in-the-blank)* so that I can create cards that hide specific parts of text for active recall.
Acceptance Criteria	Cloze syntax {{c1::hidden text}} is supported. Multiple cloze fields on one card generate multiple review cards. Preview mode shows how the card will appear during a review.

US-12	As a **content creator**, I want to *add tags to cards* so that I can filter and search cards across decks by topic.
Acceptance Criteria	Tags can be added as a comma-separated list during card creation/editing. Cards can be searched/filtered by tag. Tags are auto-suggested based on existing tags in the user's library.

3.1.4 Review Sessions

US-13	As a **learner**, I want to *start a review session* so that I can get through today's cards without thinking about what to study.
Acceptance Criteria	Dashboard shows the number of cards due for each deck. Clicking 'Study Now' enters a review session showing only due cards. New cards are also presented according to the configured daily new-card limit. The session ends when all due and new cards are reviewed.

US-14	As a **learner**, I want to *rate how well I remembered each card (Again, Hard, Good, Easy)* so that the app knows when to show it to me again.
Acceptance Criteria	Four rating buttons are displayed after revealing a card's answer: Again, Hard, Good, Easy. Each button displays the projected next review interval (e.g., '10 min', '3 days', '1 week'). The app updates the card's next review date based on the rating. Card scheduling state is persisted immediately after each rating.

US-15	As a **learner**, I want to *undo the last card rating during a review session* so that I can correct accidental mis-ratings without ruining my schedule.
Acceptance Criteria	An 'Undo' button is visible during a review session. Undo reverts the last card's FSRS state to its pre-rating values. Undo is only available for the immediately preceding card.

US-16	As a **learner**, I want to *see a summary of my review session when it ends* so that I can track my daily study performance.
Acceptance	

Criteria	Summary screen shows: total cards reviewed, number of Again/Hard/Good/Easy ratings, time spent. A retention rate for the session is displayed. A button returns the user to their dashboard.

3.1.5 Stats and Progress

US-17	As a **learner**, I want to *view my review history and retention statistics for each deck* so that I can monitor my long-term learning progress.
Acceptance Criteria	A Statistics page is accessible per deck. Charts show: daily reviews over time, card maturity distribution, forecasted review load. Overall retention rate (based on FSRS retrievability R) is displayed.

US-18	As a **learner**, I want to *view a forecast of how many cards are due in the next 30 days* so that I can plan my study schedule in advance.
Acceptance Criteria	A bar chart shows projected daily review count for the next 30 days. The forecast is recalculated after each review session.

3.1.6 Settings

US-19	As a **learner**, I want to *configure per-deck settings such as daily new card limit and maximum interval* so that I can customize my learning pace for each subject.
Acceptance Criteria	Settings panel is accessible per deck. Options include: maximum daily new cards (default 20), maximum interval (default 365 days). Changes take effect starting from the next review session.

US-20	As a **advanced user**, I want to *tweak the scheduler's internal parameters* so that it fits how my memory actually works.
Acceptance Criteria	An advanced settings section shows the 19 tuning parameters. Parameters are editable with numeric input fields. A 'Reset to defaults' button restores the published FSRS default weights. A warning explains that changing these can mess up your schedule if you're not careful.

3.2 Non-Functional Requirements

These requirements describe how the system should perform, not just what it does. They're just as important as the features.

Category	Requirement	Measurable Target
Performance	API response time for card rating (FSRS computation + DB write) under typical load.	*< 300 ms at 95th percentile*
Performance	Page initial load time (Time to Interactive) on a standard broadband connection.	*< 2 seconds*
Performance	The system must support concurrent active review sessions.	*>= 500 concurrent users*
Reliability	The application must maintain high uptime over any 30-day period.	*>= 99.5% uptime SLA*
Reliability	Scheduled card data must not be lost in the event of a server crash.	*Zero data loss; all writes are persisted before acknowledgment*
Security	All data in transit between client and server must be encrypted.	*TLS 1.2 or higher (HTTPS enforced)*
Security	User passwords must never be stored in plaintext.	*bcrypt hashing with a minimum cost factor of 12*
Security	The application must be protected against OWASP Top 10 vulnerabilities.	*Verified by a security review pre-release*
Usability	A new user must be able to complete their first review session without external help.	*Task completion rate >= 90% in usability testing*
Usability	The interface must be responsive and usable on screens >= 320px wide.	*Verified on iOS Safari, Android Chrome, and desktop browsers*
Usability	WCAG 2.1 AA accessibility compliance for all core user flows.	*0 critical accessibility violations in automated audit*
Correctness	FSRS scheduling output must match the official FSRS v5 reference implementation.	*100% match on the official FSRS test suite vectors*
Maintainability	The codebase must have sufficient test coverage for the FSRS scheduling module.	*>= 90% unit test coverage for FSRS module*
Scalability	The database schema must support incremental scaling without full migrations.	*Horizontal scaling via read replicas and connection pooling*
Portability	User data must be exportable in a standard format for long-term portability.	*.apkg and .csv export formats supported*